PRÁCTICA 2 · RA1

Pipeline de integración multifuente

CSV · API REST · JSON · Web Scraping · Pandas

M4 · Sistemes de Big Data
RA1 · Integración, procesamiento y análisis de información

Guía rápida

Duración orientativa	Herramientas	Entrega
8-10 horas	Python 3 · Pandas · Requests · BeautifulSoup · PyArrow	Notebook · Parquet · Diagrama · Informe PDF · ZIP

RUTA DE TRABAJO

Fuentes → API y scraping → Relaciones → Integración → Análisis → Validación

CHECKLIST ANTES DE ENTREGAR

- [ ] La API se pagina automáticamente y se recupera todo el catálogo.
- [ ] Las claves y cardinalidades se analizan antes de cada merge.
- [ ] La integración no multiplica filas accidentalmente.
- [ ] El dataset, el diagrama y el informe están incluidos en el ZIP.

Rubén Gasque Todolí

Mapa de la práctica

Localiza rápidamente cada fase; la numeración ya corresponde a esta versión compacta.

1. Contexto 3
2. Objetivos de aprendizaje 3
3. Criterios de evaluación relacionados 3
4. Fuentes disponibles 4
   - 4.1. Fuente A: ventas internas 4
   - 4.2. Fuente B: catálogo mediante API REST 4
   - 4.3. Fuente C: precios de competencia 5
5. Parte 1: inspección de las fuentes 5
6. Parte 2: consumo de la API REST 5
   - 6.1. La primera respuesta no necesariamente contiene todo 6
   - 6.2. Construcción del DataFrame de productos 6
7. Parte 3: web scraping 6
   - 7.1. Tratamiento del precio 7
8. Parte 4: comprender las relaciones 7
9. Parte 5: integración ventas–catálogo 7
10. Parte 6: integración con la competencia 8
    - 10.1. Construcción de un resumen de competencia 8
11. Parte 7: construcción del dataset analítico 8
    - 11.1. Variables derivadas 9
12. Parte 8: análisis de negocio 9
13. Parte 9: validación 9
14. Parte 10: exportación 10
15. Parte 11: diagrama del pipeline 10
16. Parte 12: reflexión final 11
17. Requisitos técnicos 11
18. Reproducibilidad 11
19. Contingencia de la API 12
20. Entrega 12
21. Evaluación 12
22. Pregunta final 13

1. Contexto

La empresa ficticia **ShopData S.L.** gestiona una plataforma de comercio electrónico.
La información necesaria para analizar el negocio se encuentra distribuida entre diferentes sistemas:

1. las ventas históricas de ShopData se almacenan en un fichero CSV;
2. el catálogo actual de productos se consulta mediante una API REST pública;
3. los precios observados en diferentes competidores se encuentran publicados en una página HTML.

La empresa quiere construir un único conjunto de datos que permita analizar conjuntamente la información procedente de estas tres fuentes.

Objetivo
Construir mediante Python un pipeline reproducible capaz de realizar:

CSV+API REST+HTML → Transformación → Integración → Validación → Dataset analítico

El proceso deberá poder ejecutarse automáticamente desde el principio sin modificar manualmente ninguna de las fuentes.

2. Objetivos de aprendizaje

Al finalizar la práctica deberás ser capaz de:

- extraer información desde diferentes tipos de fuentes;
- consumir una API REST utilizando Python;
- interpretar respuestas JSON;
- gestionar correctamente la paginación de una API;
- extraer información estructurada desde HTML;
- transformar datos procedentes de fuentes heterogéneas;
- identificar claves y relaciones entre datasets;
- aplicar correctamente diferentes tipos de *join*;
- detectar problemas asociados a relaciones 1:N;
- validar el resultado de una integración;
- generar variables derivadas;
- construir un dataset analítico reproducible.

3. Criterios de evaluación relacionados

Esta práctica trabaja especialmente:

- **CE 1.2.** Extrae de forma automática información y conocimiento a partir de grandes volúmenes de datos.
- **CE 1.3.** Combina diferentes fuentes y tipos de datos.
- **CE 1.4.** Construye conjuntos de datos complejos y relaciona correctamente su información.
- **CE 1.5.** Establece objetivos, prioridades y secuencia adecuadamente el proceso.

- **CE 1.6.** Selecciona e integra sistemas de información que satisfacen las necesidades del problema.

4. Fuentes disponibles

Para realizar la práctica dispones de tres fuentes diferentes.

4.1. Fuente A: ventas internas

Se proporciona el archivo:

`shopdata_sales.csv`

El fichero contiene aproximadamente 75.000 transacciones históricas de ShopData.
Entre sus variables se encuentran:

Variable	Descripción
sale_id	Identificador de la venta.
date	Fecha y hora de la transacción.
product_id	Identificador del producto.
units	Número de unidades vendidas.
unit_price	Precio unitario aplicado en la venta.
customer_country	País del cliente.
channel	Canal utilizado para realizar la compra.

El precio registrado en esta fuente representa el precio histórico aplicado en el momento de realizar la venta.

4.2. Fuente B: catálogo mediante API REST

El catálogo actual se encuentra disponible mediante la API pública:

https://dummyjson.com/products

Documentación:

https://dummyjson.com/docs/products

La API devuelve la información en formato JSON.
Los productos contienen variables como:

- `id`;
- `title`;
- `category`;
- `price`;
- `rating`;
- `stock`;
- `brand`;
- `sku`;
- `availabilityStatus`;
- otras variables adicionales.

No es necesario conservar todas las variables.
Deberás seleccionar las que consideres relevantes para el análisis.

4.3. Fuente C: precios de competencia

Se proporciona:

`competitor_prices.html`

El fichero simula una página web pública perteneciente a un comparador de precios denominado *PriceScope*.
Contiene diferentes ofertas de productos procedentes de varios competidores.
Deberás obtener su información mediante técnicas de *web scraping*.

> No se proporciona directamente un CSV con los precios.
> Tu código deberá inspeccionar y procesar el HTML para recuperar automáticamente la información.

5. Parte 1: inspección de las fuentes

Carga el fichero de ventas e inspecciona su estructura.
Como mínimo analiza:

- número de registros;
- columnas;
- tipos de datos;
- rango temporal;
- número de productos diferentes;
- distribución de los canales de venta;
- posibles valores ausentes.

Crea el DataFrame:

`df_sales`

Antes de integrar ninguna fuente debes comprender qué información contiene.

6. Parte 2: consumo de la API REST

Utiliza la librería:

`requests`

para consultar el catálogo.
Ejemplo básico:

```python
import requests

url = "https://dummyjson.com/products"

response = requests.get(url)

print(response.status_code)

data = response.json()
```

Debes comprobar:

- código HTTP recibido;
- estructura de la respuesta;
- claves principales del JSON;
- número de productos recibidos;

- número total de productos disponible.

6.1. La primera respuesta no necesariamente contiene todo

La API utiliza paginación.
La respuesta contiene información como:

- `products`;
- `total`;
- `skip`;
- `limit`.

Por defecto, una consulta al endpoint no devuelve necesariamente todos los productos existentes.
La API permite utilizar los parámetros:

`limit` y `skip`

para gestionar la paginación.

Tu código deberá recuperar automáticamente todo el catálogo necesario.
No debes asumir un número fijo de productos.
El valor de `total` deberá obtenerse de la propia respuesta de la API.

No se aceptará realizar manualmente una petición diferente para cada página.

6.2. Construcción del DataFrame de productos

Convierte los datos obtenidos en un DataFrame denominado:

`df_products`

Selecciona las columnas necesarias para poder relacionar posteriormente el catálogo con las ventas.
Analiza:

- unicidad del identificador;
- tipos de datos;
- productos sin marca u otras variables opcionales;
- productos sin stock;
- categorías existentes.

7. Parte 3: web scraping

Carga:

`competitor_prices.html`

y analiza su estructura HTML.
Puedes utilizar:

```
from bs4 import BeautifulSoup

with open("competitor_prices.html",
          encoding="utf-8") as f:
    soup = BeautifulSoup(f, "html.parser")
```

Deberás identificar automáticamente los elementos HTML que contienen las ofertas.
Extrae como mínimo:

- identificador del producto;
- competidor;
- nombre mostrado;

- precio;
- disponibilidad.

Convierte el resultado en:

`df_competitors`

7.1. Tratamiento del precio

Los precios de la página pueden no utilizar exactamente el mismo formato.
Por ejemplo:

`49,50 €`

`€ 49,50`

`49,50 EUR`

Deberás transformarlos a una variable numérica común.

8. Parte 4: comprender las relaciones

Antes de realizar ningún `merge()`, responde mediante código y explicación:

1. ¿Cuál es la clave principal de las ventas?
2. ¿Qué variable permite relacionar una venta con un producto de la API?
3. ¿Es `product_id` único en las ventas?
4. ¿Es el identificador del producto único en el catálogo?
5. ¿Es el identificador del producto único en los datos de competencia?
6. ¿Qué cardinalidad existe entre productos y ventas?
7. ¿Qué cardinalidad existe entre productos y ofertas de competencia?

Utiliza comprobaciones mediante código.
Por ejemplo:

```
df["product_id"].is_unique
df["product_id"].duplicated().sum()
df["product_id"].value_counts()
```

9. Parte 5: integración ventas–catálogo

Integra:

Ventas + Catálogo API

Debes decidir qué tipo de *join* resulta más apropiado.
No se proporciona directamente la solución.
Puedes utilizar:

```
pd.merge(...)
```

Después del *merge*, calcula:

- número de filas antes;
- número de filas después;
- número de ventas con producto encontrado;
- número de ventas sin correspondencia en el catálogo;

- porcentaje de cobertura.

Investiga los registros que no encuentran correspondencia.

> Un producto presente en el histórico de ventas no tiene por qué seguir existiendo en el catálogo actual.
> Un registro sin correspondencia no implica necesariamente un error del pipeline.

10. Parte 6: integración con la competencia

Ahora debes incorporar la tercera fuente.
Antes de hacer el *merge*, analiza nuevamente la cardinalidad.

> Un mismo producto puede aparecer varias veces en la web de competencia porque diferentes vendedores pueden ofrecerlo a precios distintos.
> Por tanto, debes pensar cuidadosamente qué ocurriría al fusionar directamente estas ofertas con las transacciones.

Compara:

- número de filas antes del *merge*;
- número de filas después;
- productos con una oferta;
- productos con varias ofertas;
- productos sin información de competencia.

Explica cualquier aumento inesperado en el número de filas.

10.1. Construcción de un resumen de competencia

Antes de incorporar la competencia al dataset analítico puede ser necesario transformar la relación 1:N.
Construye para cada producto variables como:

- número de competidores;
- precio mínimo observado;
- precio medio observado.

Puedes utilizar operaciones de agregación como:

```
groupby()
agg()
```

Justifica por qué esta transformación facilita posteriormente la integración con las ventas.

11. Parte 7: construcción del dataset analítico

Construye un DataFrame final denominado:

`df_analytical`

Deberá combinar información de:

- transacción;
- producto;
- catálogo actual;
- competencia.

11.1. Variables derivadas

Crea como mínimo:

Facturación

$$revenue = units \times unit_price$$

Diferencia con el mejor precio de competencia

$$price_difference = current_price - competitor_min_price$$

donde `current_price` representa el precio actual obtenido de la API.

Diferencia porcentual

$$price_difference_\% = \frac{current_price - competitor_min_price}{competitor_min_price} \times 100$$

Interpreta:

- valor positivo;
- valor negativo;
- valor igual a cero.

12. Parte 8: análisis de negocio

Utilizando `df_analytical`, responde mediante código a las siguientes preguntas:

1. ¿Cuáles son los 10 productos con mayor facturación histórica?
2. ¿Qué categorías generan mayor facturación?
3. ¿Qué canales concentran más ventas?
4. ¿Qué productos tienen actualmente un precio superior al menor precio observado en la competencia?
5. ¿Qué productos presentan la mayor diferencia porcentual?
6. ¿Qué porcentaje de las ventas corresponde a productos sin información de competencia?
7. ¿Existen productos vendidos históricamente que actualmente aparecen sin stock?
8. ¿Qué productos históricos no aparecen actualmente en el catálogo?

Incluye al menos:

- una operación `groupby`;
- un filtrado;
- una ordenación;
- una visualización sencilla.

13. Parte 9: validación

No basta con que el código termine sin producir errores.
Debes demostrar que la integración es coherente.
Comprueba automáticamente aspectos como:

- unicidad de `sale_id`;
- tipos de datos;
- unidades positivas;
- precios positivos;
- número esperado de filas;
- ausencia de multiplicaciones accidentales;
- coherencia de las variables derivadas.

Incluye varias comprobaciones mediante:

```
assert ...
```

o procedimientos equivalentes.
También puedes utilizar:

```
pd.merge(
    ...,
    indicator=True
)
```

para estudiar la cobertura de los *joins*.

14. Parte 10: exportación

Exporta el dataset final como:

`shopdata_analytical.parquet`

Vuelve a cargar el fichero:

```
df_check = pd.read_parquet(
    "shopdata_analytical.parquet"
)
```

y verifica que mantiene:

- número de registros;
- columnas;
- tipos principales.

15. Parte 11: diagrama del pipeline

Representa gráficamente el pipeline construido.
Debe mostrar como mínimo:

shopdata_sales.csv → Pandas
DummyJSON API → Requests + JSON
competitor_prices.html → BeautifulSoup

seguido de:

Transformación → Integración → Validación → Parquet

Puedes utilizar:

- draw.io;
- PowerPoint;
- Canva;
- Mermaid;

- otra herramienta equivalente.

16. Parte 12: reflexión final

Responde brevemente:

1. ¿Cuál de las tres fuentes fue más difícil de procesar?
2. ¿Por qué recibir un código HTTP 200 no garantiza haber descargado todo el catálogo?
3. ¿Por qué no sería recomendable relacionar los productos utilizando únicamente su nombre?
4. ¿Qué problema provoca una relación 1:N al realizar un *merge*?
5. ¿Qué diferencia existe entre un `inner join` y un `left join` en este problema?
6. ¿Por qué pueden existir ventas cuyo producto ya no aparece en el catálogo?
7. ¿Qué riesgos tendría depender del HTML de una web real?
8. ¿Por qué es importante validar el número de filas después de cada integración?

17. Requisitos técnicos

Utiliza:

- Python 3;
- Pandas;
- Requests;
- BeautifulSoup;
- PyArrow;
- Google Colab o Jupyter Notebook.

En Colab puedes instalar las dependencias necesarias mediante:

```
!pip install beautifulsoup4 pyarrow
```

18. Reproducibilidad

El notebook deberá poder ejecutarse completamente desde el principio.
No se permite:

- modificar manualmente `shopdata_sales.csv`;
- copiar manualmente productos de la API;
- copiar manualmente precios del HTML;
- crear manualmente el DataFrame de competencia;
- corregir el dataset final mediante Excel;
- realizar una petición independiente escrita manualmente para cada página de la API.

El notebook debe comportarse como un pipeline.
Si cambian los datos pero se mantiene la estructura de las fuentes, el código debería seguir funcionando sin realizar modificaciones manuales registro por registro.

19. Contingencia de la API

La práctica debe realizarse normalmente utilizando la API pública de DummyJSON.

En caso de que el servicio no esté disponible durante la sesión, el profesor podrá proporcionar una copia de respaldo de los productos necesarios.

> El fichero de respaldo solo deberá utilizarse cuando lo indique expresamente el profesor.
> El objetivo principal de la práctica es trabajar con una API REST real.

20. Entrega

La entrega deberá contener:

1. **Notebook**

   `Apellido_Nombre_P2.ipynb`

2. **Dataset final**

   `shopdata_analytical.parquet`

3. **Diagrama del pipeline**

   PDF o imagen.

4. **Informe breve**

   Máximo recomendado: 3–4 páginas.

El informe deberá resumir:

- las tres fuentes;
- decisiones de integración;
- claves y cardinalidades;
- cobertura de los *joins*;
- principales resultados;
- problemas encontrados;
- conclusiones.

Todos los archivos deberán entregarse en:

`Apellido_Nombre_P2.zip`

21. Evaluación

La práctica se evaluará sobre 10 puntos.

Apartado	Peso
Extracción e inspección de las fuentes	15 %
API REST, JSON y paginación	20 %
Web scraping	15 %
Claves, cardinalidad e integración	25 %
Validación y reproducibilidad	15 %
Análisis, diagrama y conclusiones	10 %
Total	**100 %**

Se valorará especialmente:

- recuperar correctamente el catálogo de la API;
- no asumir manualmente el número de productos;
- extraer automáticamente las ofertas del HTML;
- analizar las claves antes de realizar los *merge*;
- reconocer correctamente las relaciones 1:N;
- detectar multiplicaciones inesperadas de filas;
- justificar los tipos de *join*;
- medir la cobertura de la integración;
- construir un pipeline completamente reproducible.

22. Pregunta final

Python ejecuta:

```
pd.merge()
```

sin generar ninguna excepción.

¿Significa esto que la integración es correcta?

Justifica tu respuesta teniendo en cuenta:

- claves;
- cardinalidad;
- pérdida de registros;
- duplicación de filas;
- cobertura de la integración.